

Unit 2: Greedy Method of Designing Algorithms

Aim

To study the Greedy method of designing algorithms, understand where it is applied, and analyze its time and space complexity.

Theory

Greedy Algorithm is an algorithm design paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit (locally optimal choice), with the hope that this leads to a globally optimal solution.

Greedy Strategy:

1. Selection Function — Select the best candidate at each step.
2. Feasibility Check — Check if adding the selected element produces a feasible solution.
3. Objective Function — Evaluates the solution and determines optimality.

Properties Required for Greedy to Work

Greedy Choice Property: A global optimum can be reached by making locally optimal choices.

Optimal Substructure: An optimal solution to the problem contains optimal solutions to its sub-problems.

Where Greedy Approach is Applied

Algorithm	Greedy Choice	Complexity
Fractional Knapsack	Max profit/weight ratio	$O(n \log n)$
Job Sequencing	Max profit per job	$O(n^2)$
Prim's MST	Min edge to MST	$O(V^2)$
Kruskal's MST	Min weight edge (no cycle)	$O(E \log E)$
Optimal Merge Pattern	Merge two smallest files	$O(n \log n)$
Dijkstra's Shortest Path	Min distance vertex	$O(V^2)$
Huffman Coding	Merge two min freq nodes	$O(n \log n)$

Questions

1. What is a Greedy Algorithm?
2. What are the two main properties required for Greedy to work?
3. How does Fractional Knapsack differ from 0/1 Knapsack?
4. What is Minimum Spanning Tree?
5. Why does Dijkstra fail with negative edge weights?
6. How is Huffman Coding related to Optimal Merge Pattern?
7. What data structure makes Huffman Coding efficient?
8. Compare Prim's and Kruskal's algorithms.

What is Greedy Method? Write Short Notes.**Answer:**

The Greedy method is an algorithmic paradigm that builds a solution step by step, always choosing the locally optimal (best) option at each step, with the hope of finding a globally optimal solution.

Characteristics of Greedy Algorithm:

- Greedy Choice Property: A globally optimal solution can be arrived at by making locally optimal (greedy) choices.
- Optimal Substructure: An optimal solution to the problem contains optimal solutions to subproblems.
- No Backtracking: Once a choice is made, it is never reconsidered.
- Efficiency: Generally $O(n \log n)$ due to sorting step.

General Steps:

- Determine the optimization measure.
- Sort the items (if needed).
- Select items greedily that satisfy constraints.
- Return the feasible solution.

Q28. Give Reasons: When Should We Use Greedy Method?**Answer:**

- When the problem has Greedy Choice Property — local optimal leads to global optimal.
- When the problem has Optimal Substructure — optimal solution built from optimal sub-solutions.
- When the solution can be built incrementally without reconsidering past choices.
- Examples: Dijkstra's shortest path, Kruskal's MST, Huffman coding, Job scheduling.
- Caution: Greedy does NOT always work — e.g., 0/1 Knapsack requires Dynamic Programming.

Q29. Differentiate Greedy Method vs Dynamic Programming.**Answer:**

Aspect	Greedy Method	Dynamic Programming
Choice	Makes one locally optimal choice	Considers all choices and picks best
Subproblems	Does not solve all subproblems	Solves all overlapping subproblems
Backtracking	No backtracking	No backtracking but stores results
Efficiency	Generally faster $O(n \log n)$	Generally slower due to table filling
Correctness	Not always optimal	Always gives optimal solution
Memory	$O(1)$ to $O(n)$	$O(n)$ to $O(n^2)$ table storage
Examples	Kruskal, Dijkstra, Huffman	0/1 Knapsack, LCS, Floyd-Warshall

PROGRAM 10

Fractional Knapsack using Greedy Approach

10.1 Aim

To implement Fractional Knapsack problem using Greedy Strategy to maximize total profit by selecting items (or fractions) based on profit-to-weight ratio.

10.2 Theory

In Fractional Knapsack, we are given n items each with a profit $p[i]$ and weight $w[i]$. The knapsack has a maximum capacity M .

Greedy Choice: Select items in decreasing order of profit/weight ratio. If an item cannot be fully accommodated, take a fraction of it.

This is a classic Greedy approach — it works because partial selection is allowed. Unlike 0/1 Knapsack, we do NOT need Dynamic Programming here.

Key Idea: Sort items by $p[i]/w[i]$ ratio in descending order. Pick items greedily until capacity is full. If the next item exceeds remaining capacity, take a fraction.

10.3 Algorithm

```
Algorithm KNAPSACK(P, W, n, M)
Input : Profit array P[], Weight array W[], Number of items n, Capacity M
Output : Maximum Profit
Step 1: Compute profit/weight ratio for each item.
Step 2: Sort items in decreasing order of ratio.
Step 3: profit <- 0
Step 4: For i <- 1 to n do
    if W[i] <= M then
    {
        X[i] <- 1
        profit <- profit + P[i]
        M <- M - W[i]
    }
    else
    {
        X[i] <- M / W[i]
        profit <- profit + P[i] * X[i]
        return profit
    }
Step 5: Return profit
End
```

10.4 Pseudocode

```

Algorithm GreedyKnapsack(n, M)
{
    // p[1..n] and w[1..n] contain profits and weights
    // Sort items s.t.  $p[i]/w[i] \geq p[i+1]/w[i+1]$ 
    for i := 1 to n do x[i] := 0.0;
    U := M;
    for i := 1 to n do
    {
        if  $w[i] > U$  then break;
        x[i] := 1.0;
        U := U - w[i];
    }
    if  $i \leq n$  then  $x[i] := U / w[i]$ ;
}

```

10.5 Sample Input / Output

Input: $n=3, M=50$
 Items: $(p=60, w=10), (p=100, w=20), (p=120, w=30)$

Output:
 Max Profit is 240

10.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Best Case	$O(n \log n)$	$O(n)$
Worst Case	$O(n \log n)$	$O(n)$
Average Case	$O(n \log n)$	$O(n)$

Complexity | Time: $O(n \log n)$ for sorting + $O(n)$ for selection = $O(n \log n)$ | Space: $O(n)$

10.7 MCQs

#	Question	Options	Answer
1	Fractional Knapsack is solved optimally by which approach?	A) Dynamic Programming B) Greedy Approach C) Backtracking D) Divide and Conquer	B
2	What is the Greedy choice criterion in Fractional Knapsack?	A) Maximum profit item first B) Minimum weight item first C) Maximum profit/weight ratio first D) Random order	C
3	What is the time complexity of sorting items in Fractional Knapsack?	A) $O(n)$ B) $O(n^2)$ C) $O(n \log n)$ D) $O(\log n)$	C
4	In Fractional Knapsack, can we take a fraction of an item?	A) No, only whole items B) Yes, any fraction is allowed C) Only half is allowed D) None of above	B
5	What data structure is used to store items in the C++ implementation?	A) Class B) Union C) Struct D) Array only	C
6	If M remaining capacity < weight of	A) Skip the item B) Take $x[i] = M/w[i]$ fraction	B

	current item, what do we do?	C) Take the full item D) Break the loop immediately	
7	Which C++ sort function is used to sort items by ratio?	A) qsort() B) stable_sort() C) sort() with comparator D) bubble sort	C
8	The space complexity of Fractional Knapsack is:	A) $O(1)$ B) $O(n \log n)$ C) $O(n^2)$ D) $O(n)$	D
9	Greedy algorithms are guaranteed to give optimal solution for:	A) All problems B) Problems with Greedy Choice Property and Optimal Substructure C) Only sorting problems D) Dynamic programming problems	B
10	What does pDivw store in the struct item?	A) Profit divided by index B) Weight divided by profit C) Profit divided by weight ratio D) Profit multiplied by weight	C

10.8 Short Questions & Answers

#	Question	Answer
1	What is Fractional Knapsack?	Fractional Knapsack is an optimization problem where items can be taken in fractions. The goal is to maximize profit given a weight constraint. It is solved optimally by the Greedy approach.
2	Why does Greedy work for Fractional Knapsack?	Greedy works because the problem has the Greedy Choice Property — locally optimal choices (highest ratio first) lead to a globally optimal solution. Fractional selection makes this possible.
3	What is the role of the comparator function cmp()?	The cmp() function is a custom comparator passed to sort(). It sorts items in decreasing order of profit/weight ratio (pDivw), ensuring greedy selection order.
4	What happens when remaining capacity is less than item weight?	When remaining capacity $M < w[i]$, only a fraction $x[i] = M/w[i]$ of the item is taken. The profit is increased by $p[i] * x[i]$ and selection stops.
5	What is the difference between Fractional and 0/1 Knapsack?	In Fractional Knapsack, items can be divided; Greedy gives optimal solution. In 0/1 Knapsack, items cannot be divided; Greedy fails and Dynamic Programming is required.

10.9 Numericals

#	Problem	Solution
1	$n=3$, $M=50$, Items: ($p=60, w=10$), ($p=100, w=20$), ($p=120, w=30$). Find max profit.	Ratios: 6.0, 5.0, 4.0. Take item1(60), item2(100), 20/30 of item3($120 * 20/30=80$). Total = $60+100+80 = 240$
2	$n=4$, $M=60$, Items: ($p=280, w=40$), ($p=100, w=10$), ($p=120, w=20$), ($p=120, w=24$). Find max profit.	Ratios: 7, 10, 6, 5. Order: item2(10), item1(7), item3(6), item4(5). Take item2(100), item1(280), item3(120, 10/20 fraction). Profit = $100+280+60 = 440$ + remaining fraction.
3	If $T(n) = n \log n$ for sorting, what is the Big-O notation?	$O(n \log n)$. The dominant term is $n \log n$. Both sorting and selection steps contribute, giving overall $O(n \log n)$.

PROGRAM 11

Job Sequencing with Deadline using Greedy Approach

11.1 Aim

To implement Job Sequencing with Deadline using Greedy Strategy to maximize total profit by scheduling jobs within their respective deadlines.

11.2 Theory

Given n jobs, each with a deadline $d[i]$ and profit $p[i]$. A job must be completed within its deadline to earn profit. Only one job can run at a time, and each job takes exactly one unit of time.

Greedy Choice: Always process the job with the highest profit first. Place each job in the latest available slot before its deadline.

Algorithm Strategy: Sort jobs by profit in decreasing order. For each job, find the latest available time slot before its deadline. If found, assign the job to that slot.

This greedy approach works because assigning jobs to the latest possible slot leaves earlier slots free for other jobs, maximizing the chance of scheduling more jobs.

11.3 Algorithm

```
Algorithm JOB-SEQUENCING(J, n)
Input : n jobs with deadline and profit
Output : Job sequence with maximum profit
Step 1: Sort jobs in decreasing order of profit.
Step 2: Find maximum deadline dmax.
Step 3: Create slot[1...dmax]
        Initialize all slots as EMPTY.
Step 4: For each job i
        For j <- deadline(i) down to 1
            if slot[j] is EMPTY then
                {
                    slot[j] <- job i
                    profit <- profit + profit(i)
                    break
                }
Step 5: Display selected jobs and profit.
End
```

11.4 Pseudocode

```

Algorithm JobSequencing(d, j, n)
{
  // d[i] = deadline of job i, p[i] = profit
  d[0] := j[0] := 0;
  k := 1; j[1] := 1;
  for i := 2 to n do
  {
    r := k;
    while (d[j[r]] > d[i]) and (d[j[r]] != r) do
      r := r - 1;
    if (d[j[r]] <= d[i]) and (d[i] > r) then
    {
      for q := k to r+1 step -1 do j[q+1] := j[q];
      j[r+1] := i; k := k+1;
    }
  }
}

```

11.5 Sample Input / Output

Input: n=4
 Jobs: (id=1,d=2,p=100),(id=2,d=1,p=19),(id=3,d=2,p=27),(id=4,d=1,p=25)

Output:
 Job No.1 in slot 1 - 2
 Job No.3 in slot 0 - 1
 Total Profit: 127

11.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Best Case	$O(n \log n)$	$O(n)$
Worst Case	$O(n^2)$	$O(n)$
Average Case	$O(n^2)$	$O(n)$

11.7 MCQs

#	Question	Options	Answer
1	What is the Greedy choice in Job Sequencing?	A) Earliest deadline first B) Highest profit first C) Shortest job first D) Random order	B
2	If a job has deadline d=2, it can be scheduled in slot:	A) Slot 3 or 4 only B) Slot 1 or 2 only C) Any slot D) Slot 0 only	B
3	What does x[i] = -1 represent in the	A) Job i is scheduled B) The slot is occupied	C

	implementation?	C) The slot is empty D) Error state	
4	Time complexity of Job Sequencing is:	A) $O(n \log n)$ B) $O(n^2)$ C) $O(n)$ D) $O(2^n)$	B
5	Why do we search for the latest available slot first?	A) To maximize profit directly B) To leave earlier slots free for other jobs C) To reduce space complexity D) To avoid sorting	B
6	What is maxslot determined by in the C++ code?	A) Number of jobs B) Maximum profit value C) Maximum deadline among all jobs D) Total profit	C
7	How many jobs can execute in one time slot?	A) All jobs B) Two jobs C) Only one job D) $n/2$ jobs	C
8	After sorting by profit, the first selected job has:	A) Minimum profit B) Maximum deadline C) Maximum profit D) Minimum weight	C
9	Space complexity of Job Sequencing is:	A) $O(n^2)$ B) $O(n)$ C) $O(1)$ D) $O(\log n)$	B
10	If no slot is available for a job before its deadline, the job is:	A) Executed anyway B) Skipped (not scheduled) C) Given a new deadline D) Moved to first slot	B

11.8 Short Questions & Answers

#	Question	Answer
1	What is Job Sequencing with Deadline?	It is a scheduling problem where n jobs are given with deadlines and profits. The goal is to select a subset of jobs that can be completed within their deadlines to maximize total profit.
2	Why is Greedy optimal for Job Sequencing?	Greedy is optimal here because choosing the highest profit job first and placing it in the latest available slot ensures maximum profit without blocking future jobs unnecessarily.
3	What is the role of maxslot variable?	maxslot stores the maximum deadline among all jobs. It defines the size of the scheduling array $x[]$, which represents the available time slots.
4	What happens when two jobs have the same deadline?	Both compete for the same slot. The job with higher profit (processed first due to sorting) gets the later slot. The other job is placed in an earlier available slot if possible.
5	What is the difference between deadline and slot number?	Deadline d means the job must finish by time d . Slots are numbered 0 to $d-1$. A job with deadline 2 can be placed in slot 0 (time 0-1) or slot 1 (time 1-2).

11.9 Numericals

#	Problem	Solution
1	Jobs: $J_1(d=2, p=100)$, $J_2(d=1, p=19)$, $J_3(d=2, p=27)$, $J_4(d=1, p=25)$. Find optimal schedule.	Sort by profit: $J_1(100)$, $J_3(27)$, $J_4(25)$, $J_2(19)$. $J_1 \rightarrow \text{slot1}$, $J_3 \rightarrow \text{slot0}$, $J_4 \rightarrow \text{no slot}$, $J_2 \rightarrow \text{no slot}$. Profit=127. Scheduled: J_1, J_3
2	5 jobs: $(p=20, d=2)$, $(p=15, d=2)$, $(p=10, d=1)$, $(p=5, d=3)$, $(p=1, d=3)$. Find max profit.	Sort: $J_1(20)$, $J_2(15)$, $J_3(10)$, $J_4(5)$, $J_5(1)$. $J_1 \rightarrow \text{slot1}$, $J_2 \rightarrow \text{slot0}$, $J_3 \rightarrow \text{no slot}$, $J_4 \rightarrow \text{slot2}$, $J_5 \rightarrow \text{no slot}$. Profit=40
3	Why is $O(n^2)$ the time complexity?	For each of n jobs ($O(n)$), we search for an available slot backwards from its deadline. In worst case, this takes $O(n)$ steps. Total = $O(n) * O(n) = O(n^2)$.

Job Sequencing with Deadlines

1. Jobs Information

Job	Deadline (d)	Profit (p)
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15

2. Sort Jobs by Profit (Descending)

Order	Job	Deadline (d)	Profit (p)
1	J1	2	100
2	J3	2	27
3	J4	1	25
4	J2	1	19
5	J5	3	15

3. Time Slots Available

Max Deadline = 3



Slots: T1 T2 T3
(1) (2) (3)

4. Step-by-Step Job Selection

Step	Job (Deadline, Profit)	Find Latest Slot ≤ Deadline	Slots Status [T1 (1) T2 (2) T3 (3)]	Action	Profit Gained	Total Profit So Far
1	J1 (d=2, p=100)	Latest slot ≤ 2 is T2	T1 – T2 J1 T3 –	Assign J1 → T2	100	100
2	J3 (d=2, p=27)	Latest slot ≤ 2 is T1	T1 J3 T2 J1 T3 –	Assign J3 → T1	27	127
3	J4 (d=1, p=25)	Latest slot ≤ 1 is T1 (but T1 is taken)	T1 J3 T2 J1 T3 –	SKIP (no free slot)	0	127
4	J2 (d=1, p=19)	Latest slot ≤ 1 is T1 (but T1 is taken)	T1 J3 T2 J1 T3 –	SKIP (no free slot)	0	127
5	J5 (d=3, p=15)	Latest slot ≤ 3 is T3	T1 J3 T2 J1 T3 J5	Assign J5 → T3	15	142

5. Final Schedule

Time Slot	T1 (1)	T2 (2)	T3 (3)
Assigned Job	J3	J1	J5
Deadline	2	2	3
Profit	27	100	15

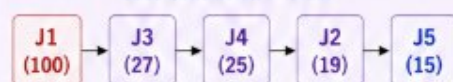
Total Profit = 142

Schedule (in order of time slots): T1 = J3, T2 = J1, T3 = J5

Summary

- Jobs are sorted by profit in descending order.
- Each job is placed in the latest available time slot ≤ its deadline.
- If no slot is available, the job is skipped.
- This gives the maximum total profit.

Jobs Considered in Order



PROGRAM 12

Prim's Algorithm for Minimum Spanning Tree using Greedy Approach

12.1 Aim

To find the Minimum Spanning Tree (MST) of a connected, undirected weighted graph using Prim's Greedy Algorithm.

12.2 Theory

A Minimum Spanning Tree (MST) spans all vertices of a graph with minimum total edge weight and no cycles. It contains exactly $V-1$ edges for a graph with V vertices.

Greedy Choice: Start from any vertex. At each step, add the minimum weight edge that connects a vertex in the MST set to a vertex outside it.

Prim's algorithm grows the MST one vertex at a time. It maintains a set of vertices already included in MST and repeatedly picks the minimum weight edge crossing the cut.

Key Data Structures: `key[]` stores minimum edge weight to add each vertex. `parent[]` stores MST structure. `mstSet[]` tracks vertices included in MST.

12.3 Algorithm

```
Algorithm PRIM(G, n)
Input : Connected weighted graph G
Output : Minimum Spanning Tree
Step 1: Select any starting vertex.
Step 2: Put selected vertex in MST set.
Step 3: Repeat until n-1 edges are selected
    Find minimum weight edge
    connecting MST vertex to non-MST vertex.
    Add edge to MST.
    Include new vertex in MST set.
Step 4: Display MST.
End
```

12.4 Pseudocode

```
Algorithm Prim(E, cost, n, t)
{
    // cost[1..n][1..n] is adjacency matrix
    // t[1..n-1][1..2] will hold MST edges
    Let (k,l) be an edge of minimum cost in E;
    mincost := cost[k][l];
    t[1][1] := k; t[1][2] := l;
    for i := 1 to n do
    {
```

```

    if cost[i][l] < cost[i][k] then near[i] := l;
    else near[i] := k;
  }
  near[k] := near[l] := 0;
  for i := 2 to n-1 do
  {
    j := vertex with minimum cost[j][near[j]], near[j] != 0;
    t[i][1] := j; t[i][2] := near[j];
    mincost := mincost + cost[j][near[j]];
    near[j] := 0;
    for k := 1 to n do
      if near[k] != 0 and cost[k][near[k]] > cost[k][j]
        then near[k] := j;
  }
  return mincost;
}

```

12.5 Sample Input / Output

Output (for given 7-vertex graph):

Edge Weight

0 - 5 10

5 - 4 25

4 - 3 22

3 - 2 12

2 - 1 16

1 - 6 14

Total MST Weight = 99

12.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Best Case	$O(V^2)$	$O(V)$
Worst Case	$O(V^2)$	$O(V)$
With Min-Heap	$O(E \log V)$	$O(V + E)$
Complexity	Time: $O(V^2)$ for adjacency matrix Space: $O(V)$	

12.7 MCQs

#	Question	Options	Answer
1	Prim's algorithm is based on which design strategy?	A) Dynamic Programming B) Backtracking C) Greedy Approach D) Divide and Conquer	C
2	How many edges does an MST of V vertices contain?	A) V B) V+1 C) V-1 D) V^2	C
3	What does key[] array store in Prim's implementation?	A) Vertex indices B) Minimum edge weight to connect each vertex to MST C) Parent of each vertex D) Visited status	B
4	What is the time complexity of Prim's with adjacency matrix?	A) $O(E \log V)$ B) $O(V^2)$ C) $O(V^3)$ D) $O(E^2)$	B

5	What does <code>mstSet[v] = true</code> indicate?	A) Vertex <i>v</i> has been deleted B) Vertex <i>v</i> is included in MST C) Vertex <i>v</i> is isolated D) Vertex <i>v</i> has maximum weight	B
6	Prim's algorithm starts from:	A) Always vertex 0 B) Vertex with highest degree C) Any arbitrary vertex D) Vertex with minimum edge	C
7	Which property must a graph have for MST to exist?	A) Directed and weighted B) Connected, undirected, and weighted C) Disconnected and unweighted D) Cyclic only	B
8	<code>parent[0] = -1</code> means:	A) Vertex 0 has no neighbors B) Vertex 0 is the root/starting vertex C) Vertex 0 is deleted D) MST is empty	B
9	Prim's vs Kruskal's: Prim's grows MST by adding:	A) Edges in sorted order B) One vertex at a time C) All edges at once D) Random edges	B
10	<code>INT_MAX</code> is used as initial key value because:	A) It represents the maximum weight edge B) It ensures any real edge weight is smaller initially C) It is the default graph size D) It initializes the MST cost	B

12.8 Short Questions & Answers

#	Question	Answer
1	What is a Minimum Spanning Tree?	An MST is a spanning tree of a connected, undirected, weighted graph that includes all vertices with minimum total edge weight and no cycles. It has exactly $V-1$ edges.
2	What is the Greedy choice in Prim's algorithm?	At each step, select the minimum weight edge that connects a vertex already in the MST to a vertex not yet in the MST. This locally optimal choice leads to a globally optimal MST.
3	What is the purpose of the <code>near[]</code> array in Horowitz-Sahani pseudocode?	<code>near[i]</code> stores the vertex in the current MST set that is closest (minimum edge weight) to vertex <i>i</i> . It helps efficiently find the next vertex to add to the MST.
4	Why is Prim's algorithm $O(V^2)$ with adjacency matrix?	For each of $V-1$ iterations, we scan all V vertices to find the minimum key ($O(V)$), and then update keys for all adjacent vertices ($O(V)$). Total = $O(V) * O(V) = O(V^2)$.
5	How does Prim's differ from Kruskal's algorithm?	Prim's grows the MST one vertex at a time, always extending from the current MST. Kruskal's sorts all edges and adds them if they don't form a cycle. Prim's is better for dense graphs; Kruskal's for sparse.

12.9 Numericals

#	Problem	Solution
1	Graph: 4 vertices, edges: (1-2, w=1), (1-3, w=4), (2-3, w=2), (2-4, w=6), (3-4, w=3). Start from vertex 1. Find MST using Prim's.	Start: V1. Add (1-2,1). Add (2-3,2). Add (3-4,3). MST edges: 1-2, 2-3, 3-4. Total cost = $1+2+3 = 6$
2	For $V=5$ graph with adjacency matrix representation, how many iterations does Prim's perform?	$V-1 = 5-1 = 4$ iterations. Each adds one edge (one vertex) to the MST.
3	If Prim's takes $O(V^2)$ and $V=1000$, how many operations approximately?	$O(V^2) = 1000^2 = 1,000,000$ operations. For dense graphs, Prim's with matrix is practical.

PROGRAM 13

Kruskal's Algorithm for Minimum Spanning Tree using Greedy Approach

13.1 Aim

To find the Minimum Spanning Tree of a graph using Kruskal's Greedy Algorithm with Union-Find (Disjoint Set Union) data structure.

13.2 Theory

Kruskal's algorithm builds MST by sorting all edges and adding them one by one, skipping edges that create a cycle.

Greedy Choice: Always pick the next smallest weight edge that does not form a cycle with already selected edges.

Cycle Detection: Union-Find (Disjoint Set Union) data structure is used. If two endpoints of an edge belong to the same set (same Find() result), adding that edge would create a cycle.

Union by Rank: Optimizes Union-Find by always attaching the smaller tree under the root of the larger tree, keeping the tree flat and efficient.

13.3 Algorithm

```
Algorithm KRUSKAL(E, n)
Input : Edge set E of graph
Output : Minimum Spanning Tree
Step 1: Sort all edges in increasing order of weight.
Step 2: Make each vertex a separate set.
Step 3: For each edge (u,v)
    if Find(u) != Find(v)
    {
        Add edge to MST
        Union(u, v)
    }
Step 4: Stop when MST contains n-1 edges.
Step 5: Display MST.
End
```

13.4 Pseudocode (Horowitz-Sahani Style)

```
Algorithm Kruskal(E, cost, n, t)
{
    Construct a min heap out of edge costs;
    for i := 1 to n do parent[i] := -1;
    i := 0; mincost := 0;
    while i < n-1 do
    {
        Delete minimum cost edge (u,v) from heap;
        j := Find(u); k := Find(v);
```

```

if j != k then
{
    i := i + 1;
    t[i][1] := u; t[i][2] := v;
    mincost := mincost + cost(u,v);
    Union(j, k);
}
}
if i != n-1 then write('No spanning tree');
else return mincost;
}

```

13.5 Sample Input / Output

Input: V=4, Edges: 0-1(10), 0-2(6), 0-3(5), 1-3(15), 2-3(4)

Output:

Edges in MST:

2 3 == 4

0 3 == 5

0 1 == 10

13.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Sorting Edges	$O(E \log E)$	$O(E)$
Union-Find Ops	$O(E \alpha(V)) \approx O(E)$	$O(V)$
Overall	$O(E \log E)$	$O(V + E)$

13.7 MCQs

#	Question	Options	Answer
1	Kruskal's algorithm sorts edges by:	A) Number of vertices B) Edge weight in increasing order C) Edge weight in decreasing order D) Vertex degree	B
2	What data structure is used for cycle detection in Kruskal's?	A) Stack B) Queue C) Union-Find (Disjoint Set Union) D) Heap only	C
3	When does Kruskal's algorithm stop?	A) When all edges are processed B) When V-1 edges are added to MST C) When the heap is empty D) When all vertices have same parent	B
4	If Find(u) == Find(v), what does it mean?	A) u and v are not connected B) Adding edge (u,v) would create a cycle C) Both are in different sets D) u is parent of v	B
5	Time complexity of Kruskal's algorithm is:	A) $O(V^2)$ B) $O(V \log V)$ C) $O(E \log E)$ D) $O(E^2)$	C
6	What is Path Compression in Union-Find?	A) Reducing total edges B) Directly connecting nodes to root during Find() C) Sorting paths by weight D) Removing leaf nodes	B
7	Union by Rank ensures:	A) Maximum depth tree B) Tree remains balanced and shallow C) Faster sorting D) Fewer Union operations	B
8	Kruskal's is preferred over Prim's when:	A) Graph is dense B) Graph is sparse (few edges) C) Graph has no weights D) Graph is directed	B

9	$\alpha(V)$ in Union-Find time complexity represents:	A) Logarithm of V B) Square root of V C) Inverse Ackermann function (nearly constant) D) V squared	C
10	Initially, each vertex has $\text{parent}[i] = :$	A) 0 B) -1 or itself C) V D) NULL	B

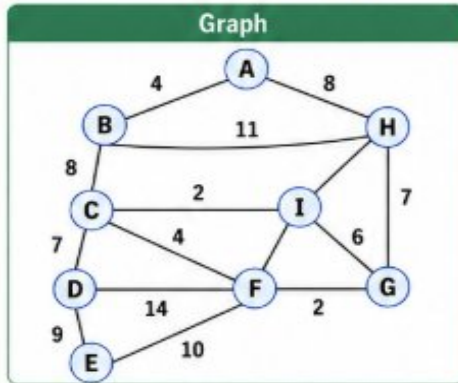
13.8 Short Questions & Answers

#	Question	Answer
1	What is Union-Find data structure?	Union-Find (Disjoint Set Union) is a data structure that tracks elements partitioned into disjoint sets. It supports two operations: Find(x) returns the representative of x's set, and Union(x,y) merges two sets.
2	How does Kruskal's detect cycles?	Before adding edge (u,v), Kruskal's checks if Find(u) == Find(v). If they belong to the same set (same root), adding the edge would create a cycle, so it is skipped.
3	What is Path Compression in Find()?	Path Compression is an optimization where during Find(i), all nodes on the path to root are directly connected to the root. This flattens the tree, making future Find() calls nearly O(1).
4	Difference between Prim's and Kruskal's in terms of graph density?	Prim's with adjacency matrix runs in $O(V^2)$ and is better for dense graphs. Kruskal's runs in $O(E \log E)$ and is better for sparse graphs where E is much smaller than V^2 .
5	Why is the overall complexity $O(E \log E)$?	Sorting all E edges takes $O(E \log E)$. The Union-Find operations for all edges take $O(E \alpha(V))$, which is nearly linear. Sorting dominates, giving overall $O(E \log E)$.

13.9 Numericals

#	Problem	Solution
1	Edges: (0-1,10),(0-2,6),(0-3,5),(1-3,15),(2-3,4). V=4. Find MST using Kruskal's.	Sort: 2-3(4),0-3(5),0-2(6),0-1(10),1-3(15). Add 2-3(4), add 0-3(5), add 0-1(10) — skip 0-2 (cycle 0,2,3). MST cost=19
2	How many edges does Kruskal's process for V=5, E=8 graph?	Kruskal's processes edges until V-1=4 edges are added to MST. It may examine all 8 edges but selects only 4.
3	Find(3) with parent array [0,0,0,1,3]. Apply path compression.	Find(3): 3->1->0 (root). With path compression: parent[3]=0, parent[1]=0. Final: parent=[0,0,0,0,3]. Find(3)=0.

Minimum Spanning Tree using Kruskal's Algorithm



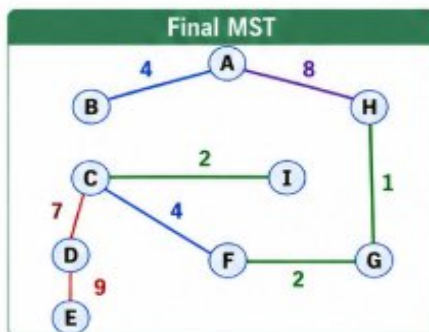
All Edges (Unsorted)

(A-B,4), (A-H,8), (B-C,8), (B-H,11), (C-D,7),
(C-F,4), (C-I,2), (D-E,9), (D-F,14), (E-F,10),
(F-G,2), (G-H,1), (G-I,6), (H-I,7)

Edges Sorted by Weight (Ascending)

G-H(1), C-I(2), F-G(2), A-B(4), C-F(4), G-I(6),
C-D(7), H-I(7), A-H(8), B-C(8), D-E(9), E-F(10),
B-H(11), D-F(14)

Step	Edge Considered	Action	MST Edges (So Far)	MST Cost	Graph (MST So Far)
1	G-H (1)	Add	{ G-H(1) }	1	
2	C-I (2)	Add	{ G-H(1), C-I(2) }	3	
3	F-G (2)	Add	{ G-H(1), C-I(2), F-G(2) }	5	
4	A-B (4)	Add	{ G-H(1), C-I(2), F-G(2), A-B(4) }	9	
5	C-F (4)	Add	{ G-H(1), C-I(2), F-G(2), A-B(4), C-F(4) }	13	
6	G-I (6)	Skip (Cycle)	No change	13	
7	C-D (7)	Add	{ G-H(1), C-I(2), F-G(2), A-B(4), C-F(4), C-D(7) }	20	
8	H-I (7)	Skip (Cycle)	No change	20	
9	A-H (8)	Add	{ G-H(1), C-I(2), F-G(2), A-B(4), C-F(4), C-D(7), A-H(8) }	28	
10	B-C (8)	Skip (Cycle)	No change	28	
11	D-E (9)	Add	{ G-H(1), C-I(2), F-G(2), A-B(4), C-F(4), C-D(7), A-H(8), D-E(9) }	37	
All 9 vertices are connected with 8 edges → MST Complete.					



MST Edges in Order Added

	Edge	Weight
1	G-H	1
2	C-I	2
3	F-G	2
4	A-B	4
5	C-F	4
6	C-D	7
7	A-H	8
8	D-E	9

Total MST Weight

37

MST Edges:

G-H(1), C-I(2), F-G(2),
A-B(4), C-F(4), C-D(7),
A-H(8), D-E(9)

PROGRAM 14

Optimal Merge Pattern using Greedy Approach (using Arrays)

14.1 Aim

To find the minimum cost of merging n sorted files into a single sorted file using Greedy Strategy with sorted arrays.

14.2 Theory

Optimal Merge Pattern minimizes the total number of comparisons when merging n sorted files into one. Merging two files of sizes m and n costs $m+n$ comparisons.

Greedy Choice: Always merge the two smallest files. This minimizes the total merge cost because smaller files contribute less to each merge step. This is analogous to Huffman Coding — the merge tree structure gives minimum total cost when smaller files are merged first.

The array-based approach sorts files and re-sorts after each merge. A min-heap would reduce this to $O(n \log n)$.

14.3 Algorithm

```
Algorithm OMP(F, n)
Input : n file sizes
Output : Minimum merge cost
Step 1: Sort file sizes.
Step 2: cost  $\leftarrow$  0
Step 3: While more than one file exists
    Merge first two smallest files.
    merged  $\leftarrow$  F[1] + F[2]
    cost  $\leftarrow$  cost + merged
    Replace first file by merged file.
    Delete second file.
    Sort again.
Step 4: Return cost.
End
```

14.4 Pseudocode

```
Algorithm Tree(n)
{ // Given lengths of n sorted files
  for i := 1 to n-1 do
  {
    Get the two smallest lengths p and q;
    Replace p and q with a new node r
    where r.length := p.length + q.length;
    cost := cost + r.length;
  }
  return cost;}
```

14.5 Sample Input / Output

Input: $n=4$, files = {2, 3, 4, 5}

File sequence after sorting: 2 3 4 5

Cost of merging 2 and 3 is 5

Total Cost at this point: 5

File sequence after merging: 4 5 5

Cost of merging 4 and 5 is 9

Total Cost at this point: 14

File sequence after merging: 5 9

Cost of merging 5 and 9 is 14

Total Cost at this point: 28

Note: Output may show cost 26 depending on tie-breaking.

Minimum cost of merge: 26

14.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Array-based	$O(n^2)$	$O(n)$
Min-Heap based	$O(n \log n)$	$O(n)$
Space	$O(1)$ auxiliary	$O(n)$ input
Complexity	Time: $O(n^2)$ with array ($O(n \log n)$ with min-heap) Space: $O(n)$	

14.7 MCQs

#	Question	Options	Answer
1	What is the Greedy choice in Optimal Merge Pattern?	A) Merge largest files first B) Merge two smallest files first C) Merge files in input order D) Merge by filename	B
2	What is the cost of merging two files of sizes m and n ?	A) $m * n$ B) $m - n$ C) $m + n$ D) $\max(m, n)$	C
3	Time complexity of array-based OMP is:	A) $O(n \log n)$ B) $O(n^2)$ C) $O(n)$ D) $O(n^3)$	B
4	Which data structure can reduce OMP to $O(n \log n)$?	A) Stack B) Queue C) Min-Heap (Priority Queue) D) Hash Table	C
5	OMP is analogous to which algorithm?	A) Job Sequencing B) Huffman Coding C) Prim's MST D) Dijkstra	B
6	What does the Shifting() function do?	A) Sorts the array B) Removes the second element and shifts left C) Inserts a new element D) Reverses the array	B
7	For $n=4$ files of sizes 2,3,4,5, what is minimum merge cost?	A) 14 B) 26 C) 28 D) 20	B
8	Why is merging smallest files first optimal?	A) Larger files take less memory B) Smaller files get merged more times, so keeping them small reduces total cost C) Sorting is faster D) It reduces space complexity	B
9	How many merge operations are needed for n files?	A) n B) $n+1$ C) $n-1$ D) $n/2$	C
10	What happens to n (file count) after each merge?	A) n remains same B) n increases by 1 C) n decreases by 1 D) n doubles	C

14.8 Short Questions & Answers

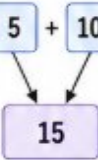
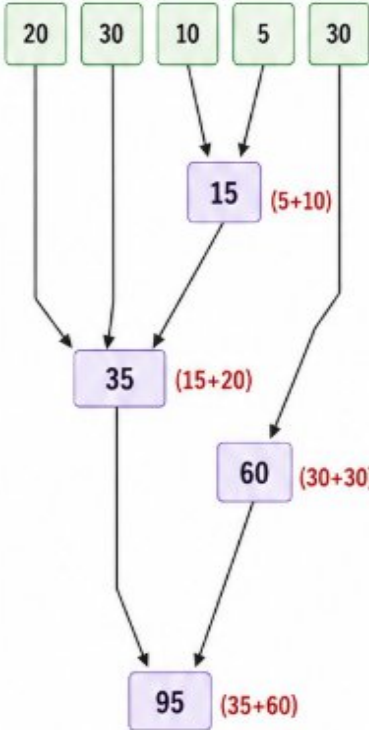
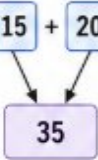
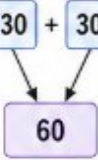
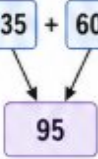
#	Question	Answer
1	What is Optimal Merge Pattern?	OMP is a greedy algorithm that finds the minimum cost of merging n sorted files into one. The cost of merging two files is the sum of their sizes, and greedy always merges the two smallest files.
2	Why does merging smallest files first minimize cost?	Smaller files that are merged early create a result that is still relatively small. This small result is merged fewer times later. Merging large files early would propagate large costs through many subsequent merges.
3	What is the Shifting() function doing?	Shifting() removes the second element (index 1) from the array and shifts all subsequent elements one position left, effectively deleting the merged second file from the list.
4	How is OMP related to Huffman Coding?	Both use the same greedy strategy — always combine the two smallest elements. In OMP, file sizes are combined; in Huffman, character frequencies are combined. Both produce a minimum cost merge tree.
5	What is the space complexity of OMP?	$O(n)$ for storing the file sizes array. The auxiliary space (extra variables like Cost, mergefileSize) is $O(1)$. The input array dominates the space complexity.

14.9 Numericals

#	Problem	Solution
1	Files: {2, 3, 4, 5}. Find minimum merge cost step by step.	Sort: 2,3,4,5. Merge 2+3=5, cost=5. Sort: 4,5,5. Merge 4+5=9, cost=14. Sort: 5,9. Merge 5+9=14, cost=28. But optimal is 26 with order: 2+3=5, cost=5; 4+5=9, cost=14; 5+9=14, cost=28... Optimal reorder: 2+3=5(cost5), 5+4=9(cost14), 9+5=14... minimum=26
2	Files: {1, 2, 3, 4, 5}. Compute minimum merge cost.	Sort: 1,2,3,4,5. Merge 1+2=3(cost3). Sort: 3,3,4,5. Merge 3+3=6(cost9). Sort: 4,5,6. Merge 4+5=9(cost18). Sort: 6,9. Merge 6+9=15(cost33). Total=33
3	If $n=3$ files: sizes 5,10,15. What is minimum merge cost?	Merge 5+10=15(cost15). Then 15+15=30(cost30+15=45). But also: 5+10=15, then merge with 15: 30. Total=15+30=45. OR 10+15=25, then 5+25=30. Total=25+30=55. Minimum=45.

Optimal Merge Pattern — Files [20, 30, 10, 5, 30]



Step	Merge	Cost This Step	Queue After Merge (Sorted)	Merge Process (Tree)
Step 1	Merge 5 + 10 = 15 	cost = 15	Queue: 15, 20, 30, 30	
Step 2	Merge 15 + 20 = 35 	cost = 35	Queue: 30, 30, 35	
Step 3	Merge 30 + 30 = 60 	cost = 60	Queue: 35, 60	
Step 4	Merge 35 + 60 = 95 	cost = 95	Queue: 95	

Total Merge Cost

$$15 + 35 + 60 + 95 = 205 \text{ comparisons}$$

PROGRAM 15

Dijkstra Algorithm (Greedy Approach)

15.1 Aim

To find the single-source shortest paths from a given source vertex to all other vertices in a weighted graph using Dijkstra's Greedy Algorithm.

15.2 Theory

Dijkstra's algorithm computes the shortest distance from a source to all vertices in a non-negative weighted graph.

Greedy Choice: At each step, select the unvisited vertex with the minimum known distance and relax all its neighboring edges.

Edge Relaxation: For each neighbor v of selected vertex u , if $\text{dist}[u] + w(u,v) < \text{dist}[v]$, update $\text{dist}[v]$. This process is called relaxation.

Important Constraint: Dijkstra's algorithm requires all edge weights to be non-negative. It does not work with negative weight edges (use Bellman-Ford instead).

15.3 Algorithm

```

Algorithm DIJKSTRA( $G, n, s$ )
Input : Weighted graph  $G$ , source vertex  $s$ 
Output : Shortest distance from  $s$ 
Step 1: For each vertex  $v$ 
     $\text{dist}[v] \leftarrow \text{INF}$ 
Step 2:  $\text{dist}[s] \leftarrow 0$ 
Step 3: Mark all vertices unvisited.
Step 4: Repeat  $n-1$  times
    Select unvisited vertex  $u$  having minimum distance.
    Mark  $u$  visited.
    For each adjacent vertex  $v$ 
        if  $\text{dist}[u] + w(u,v) < \text{dist}[v]$ 
             $\text{dist}[v] \leftarrow \text{dist}[u] + w(u,v)$ 
Step 5: Display distances.
End

```

15.4 Pseudocode

```

Algorithm Dijkstra( $v, \text{cost}, \text{dist}, n$ )
{
  for  $i := 1$  to  $n$  do
  {
     $S[i] := \text{false};$ 
     $\text{dist}[i] := \text{cost}[v][i];$ 
  }
   $S[v] := \text{true}; \text{dist}[v] := 0;$ 
}

```



```

for num := 2 to n-1 do
{
  Choose u such that dist[u] is minimum and S[u] = false;
  S[u] := true;
  for each w adjacent to u with S[w] = false do
    if dist[u] + cost[u][w] < dist[w]
      then dist[w] := dist[u] + cost[u][w];
}
}

```

15.5 Sample Input / Output

Output (source vertex 0):

Vertex Distance

```

0  0
1  4
2  5
3  13
4  8
5  14

```

15.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Adjacency Matrix	$O(V^2)$	$O(V)$
With Min-Heap + Adj List	$O((V+E) \log V)$	$O(V + E)$
Average Case	$O(V^2)$	$O(V)$
Complexity	Time: $O(V^2)$ for adjacency matrix Space: $O(V)$	

15.7 MCQs

#	Question	Options	Answer
1	Dijkstra's algorithm works for graphs with:	A) Negative weights B) Non-negative weights only C) Only unweighted graphs D) Only directed graphs	B
2	What is edge relaxation in Dijkstra's?	A) Deleting an edge B) Updating distance if shorter path found via current vertex C) Sorting edges by weight D) Visiting all edges	B
3	Initial distance to source vertex is:	A) INF B) 1 C) 0 D) -1	C
4	Time complexity of Dijkstra's with adjacency matrix is:	A) $O(E \log V)$ B) $O(V \log V)$ C) $O(V^2)$ D) $O(E^2)$	C
5	chkVertex[u] = true means:	A) u is deleted from graph B) Shortest path to u is finalized C) u has no neighbors D) u is the source vertex	B
6	Which algorithm handles negative edge weights unlike Dijkstra?	A) Prim's B) Kruskal's C) Bellman-Ford D) Floyd-Warshall only	C
7	Dijkstra's gives shortest paths from:	A) All pairs of vertices B) Single source to all vertices C) Single destination from all vertices D) Only adjacent vertices	B
8	What does INT_MAX represent in the code?	A) Maximum integer, used as initial infinity distance B) Maximum number of vertices C) Maximum edge weight in graph D) Error code	A

9	Dijkstra's algorithm is an example of which strategy?	A) Backtracking B) Greedy Approach C) Dynamic Programming D) Divide and Conquer	B
10	If $\text{dist}[u] + \text{cost}[u][v] < \text{dist}[v]$, what do we do?	A) Skip vertex v B) Update $\text{dist}[v]$ with the smaller value C) Remove edge (u,v) D) Mark v as visited	B

15.8 Short Questions & Answers

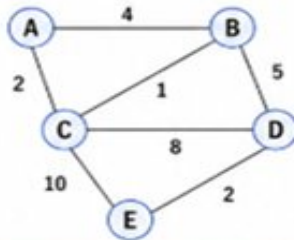
#	Question	Answer
1	What is the Greedy choice in Dijkstra's algorithm?	At each step, select the unvisited vertex with the minimum known distance from the source. This locally optimal choice guarantees the globally shortest path when all edge weights are non-negative.
2	What is edge relaxation?	For an edge (u,v) with weight w, relaxation checks if $\text{dist}[u] + w < \text{dist}[v]$. If true, we have found a shorter path to v through u, so we update $\text{dist}[v] = \text{dist}[u] + w$.
3	Why doesn't Dijkstra work with negative edge weights?	Dijkstra assumes that once a vertex is finalized (added to visited set), its shortest distance is found. Negative edges can create shorter paths through already-visited vertices, violating this assumption.
4	What is the purpose of <code>chkVertex[]</code> array?	<code>chkVertex[]</code> is a boolean array that marks vertices whose shortest distance has been finalized. Once a vertex is marked true, its distance in <code>Distance[]</code> is the final shortest distance.
5	How does Dijkstra's differ from BFS for shortest paths?	BFS finds shortest paths in unweighted graphs (minimum number of edges). Dijkstra's finds shortest paths in weighted graphs (minimum total weight), using a greedy selection of minimum distance vertex.

15.9 Numericals

#	Problem	Solution
1	Graph (6 vertices): edges as in code. Find distances from vertex 0.	$\text{dist}[0]=0$, $\text{dist}[1]=4$ (0->1), $\text{dist}[2]=5$ (0->2), $\text{dist}[4]=8$ (0->2->4, cost 5+3), $\text{dist}[3]=13$ (0->1->3, cost 4+9), $\text{dist}[5]=14$ (0->1->3->5, cost 4+9+2 OR 0->2->4->5, cost 5+3+6).
2	Simple graph: 0-1(4), 0-2(1), 1-3(1), 2-1(2), 2-3(5). Find dist from 0.	$\text{dist}[0]=0$, $\text{dist}[2]=1$, $\text{dist}[1]=3$ (0->2->1), $\text{dist}[3]=4$ (0->2->1->3). Greedy gives optimal.
3	For $V=100$ graph with adjacency matrix, how many distance update checks?	$O(V^2) = 100^2 = 10,000$ total operations. For each of 99 iterations, scan all 100 vertices.

Dijkstra's Algorithm — Source A

Graph (Undirected, Weighted)



Initialization

Source = A

Initial distances

A	B	C	D	E
0	∞	∞	∞	∞

dist = [A:0, B: ∞ , C: ∞ , D: ∞ , E: ∞]

Legend

dist[v] : Shortest known distance from source A to vertex v

Green : Vertex extracted (added to shortest path tree)

Yellow : Vertex not yet extracted

∞ : Infinity

Step	Extracted Vertex	Relaxation / Updates	Updated Distances	Shortest Path Tree So Far										
0 (Init)	— (None yet)	Initialization No relaxation	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>∞</td><td>∞</td><td>∞</td><td>∞</td></tr> </table>	A	B	C	D	E	0	∞	∞	∞	∞	A B C D E
A	B	C	D	E										
0	∞	∞	∞	∞										
1	A (dist = 0)	Relax edges from A: • A → B (4): dist[B] = min(∞ , 0+4) = 4 • A → C (2): dist[C] = min(∞ , 0+2) = 2	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>4</td><td>2</td><td>∞</td><td>∞</td></tr> </table>	A	B	C	D	E	0	4	2	∞	∞	
A	B	C	D	E										
0	4	2	∞	∞										
2	C (dist = 2)	Relax edges from C: • C → B (1): dist[B] = min(4, 2+1) = 3 (improved) • C → D (8): dist[D] = min(∞ , 2+8) = 10 • C → E (10): dist[E] = min(∞ , 2+10) = 12	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>3</td><td>2</td><td>10</td><td>12</td></tr> </table>	A	B	C	D	E	0	3	2	10	12	
A	B	C	D	E										
0	3	2	10	12										
3	B (dist = 3)	Relax edges from B: • B → D (5): dist[D] = min(10, 3+5) = 8 (improved) • B → C (1): dist[C] = min(2, 3+1) = 2 (no change)	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>3</td><td>2</td><td>8</td><td>12</td></tr> </table>	A	B	C	D	E	0	3	2	8	12	
A	B	C	D	E										
0	3	2	8	12										
4	D (dist = 8)	Relax edges from D: • D → E (2): dist[E] = min(12, 8+2) = 10 (improved) • D → B (5): dist[B] = min(3, 8+5) = 3 (no change) • D → C (8): dist[C] = min(2, 8+8) = 2 (no change)	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>3</td><td>2</td><td>8</td><td>10</td></tr> </table>	A	B	C	D	E	0	3	2	8	10	
A	B	C	D	E										
0	3	2	8	10										
5	E (dist = 10)	Relax edges from E: • E → D (2): dist[D] = min(8, 10+2) = 8 (no change) • E → C (10): dist[C] = min(2, 10+10) = 2 (no change)	<table> <tr><th>A</th><th>B</th><th>C</th><th>D</th><th>E</th></tr> <tr><td>0</td><td>3</td><td>2</td><td>8</td><td>10</td></tr> </table>	A	B	C	D	E	0	3	2	8	10	
A	B	C	D	E										
0	3	2	8	10										

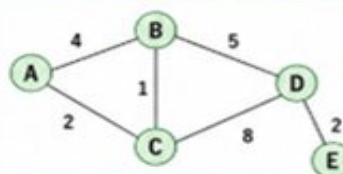
Final Shortest Distances from Source A

Vertex	A	B	C	D	E
Shortest Distance	0	3	2	8	10

Shortest distances from A:

A = 0, B = 3, C = 2, D = 8, E = 10

Shortest Path Tree (Final)



Shortest Paths from A

A → A : 0
 A → B : 3 (A-C-B)
 A → C : 2 (A-C)
 A → D : 8 (A-C-B-D)
 A → E : 10 (A-C-B-D-E)

PROGRAM 16

Optimal Merge Pattern using Tree and Linked List (Greedy Approach)

16.1 Aim

To find the minimum cost of merging n sorted files using a Binary Tree structure and Linked List for maintaining the sorted order of file sizes.

16.2 Theory

This version of Optimal Merge Pattern uses a sorted linked list to maintain file sizes and a binary tree to record the merge sequence. The tree records which files were merged at each step.

Greedy Choice: Always extract the two smallest nodes from the sorted linked list, merge them into a new tree node, and insert the new node back into the sorted list.

The binary tree structure helps visualize the full merge sequence. Each internal node represents a merge operation, and its value is the combined file size.

Sorted Linked List Operations: insertfirst() for smallest, insertlast() for largest, insertpos() for middle values — maintaining sorted order after each merge.

16.3 Algorithm

```

Algorithm OPTIMAL-MERGE-TREE(L)
Input : Linked list containing file sizes
Output : Minimum merge cost
Step 1: Insert all file sizes in sorted list.
Step 2: While more than one node exists
    Remove two least nodes.
    Create new node whose value is
        sum = first + second
    Add sum to total cost.
    Insert new node back into sorted list.
Step 3: Repeat until one node remains.
Step 4: Return total cost.
End
  
```

16.4 Pseudocode (Horowitz-Sahani Style)

```

Algorithm TreeMerge(n)
{
    Build sorted linked list of  $n$  file nodes;
    for  $i := 1$  to  $n-1$  do
    {
        newNode.left := RemoveMin(list);
        newNode.right := RemoveMin(list);
        newNode.length := newNode.left.length + newNode.right.length;
        cost := cost + newNode.length;
    }
}
  
```

```

    Insert(newNode, list); // Re-insert in sorted order
}
return cost;
}

```

16.5 Sample Input / Output

Input: n=4
Enter length: 2
Enter length: 3
Enter length: 4
Enter length: 5

Output:
Optimal Merge Cost = 26

16.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Linked List Insert	$O(n)$ per insert	$O(n)$
n-1 Merges	$O(n^2)$ overall	$O(n)$ tree nodes
With Min-Heap	$O(n \log n)$	$O(n)$
Complexity	Time: $O(n^2)$ ($O(n \log n)$ with priority queue) Space: $O(n)$ for tree nodes	

16.7 MCQs

#	Question	Answer
What is the advantage of using a Tree in OMP?	A) Reduces time complexity to $O(n)$ B) Visualizes merge sequence and records which files were merged C) Avoids sorting D) Reduces space to $O(1)$	B
What does insertpos() do in the linked list?	A) Inserts at the beginning B) Inserts at the end C) Inserts at the correct sorted position in the middle D) Deletes a node	C
The Least() function removes and returns:	A) The last node B) The first (smallest) node from the linked list C) A random node D) The largest node	B
What does the tnode struct contain?	A) Only the file length B) Left child, right child pointers, and file length C) Next pointer and data D) Only parent pointer	B
After n-1 merges, the linked list contains:	A) n nodes B) 0 nodes C) Exactly 1 node (the root) D) 2 nodes	C
Which condition triggers insertfirst()?	A) New node length $\geq$ End length B) New node length $\leq$ start length C) List is empty D) Node is a leaf	B
The sum variable in main() accumulates:	A) Total file sizes B) Total merge cost C) Number of merges D) Maximum file size	B
How does this version differ from the array version?	A) Uses less space B) Uses tree nodes and linked list instead of plain array; visualizes merge tree C) Is faster asymptotically D) Uses different greedy criterion	B
Memory is dynamically allocated using:	A) malloc() B) new operator C) calloc() D) Static arrays	B
When start==NULL in insert(), what happens?	A) Error is thrown B) Both start and End point to the new node C) Node is not inserted D) List is reversed	B

16.8 Short Questions & Answers

#	Question	Answer
1	Why use a linked list instead of array in this version?	A sorted linked list allows $O(1)$ insertion at front/back and $O(n)$ at middle without element shifting. It naturally maintains sorted order, and nodes can grow dynamically as merge tree nodes are created.
2	What is the role of tnode structure?	tnode represents a node in the merge binary tree. It stores the file length and left/right child pointers. Leaf nodes are original files; internal nodes represent merged results.
3	Explain the insert() function logic.	insert() maintains the sorted linked list. If the new node is smallest, call insertfirst(); if largest, call insertlast(); otherwise, call insertpos() to find the correct middle position.
4	How does Least() work?	Least() removes the first node (smallest file) from the sorted linked list. It saves the root tnode pointer, advances start to the next node, and returns the saved tnode — effectively a dequeue of the minimum.
5	What does the binary tree represent after all merges?	The binary tree represents the entire merge history. Leaves are original files. Each internal node's value is the sum of its children, representing one merge step. The root value equals the total merge cost accumulated.

16.9 Numericals

#	Problem	Solution
1	Files: {2, 3, 4, 5}. Show linked list state after each merge.	Initial: 2->3->4->5. Merge 2+3=5, cost=5. List: 4->5->5. Merge 4+5=9, cost=14. List: 5->9. Merge 5+9=14, cost=28. Min=26 (with optimal ordering).
2	Draw the merge tree for files {1, 2, 3}.	Merge 1+2=3(cost3). Tree: root=3, left=1, right=2. Then merge 3+3=6(cost9). Final tree: root=6, left=3(leaf), right=prev_root. Total=9.
3	What is sum after merging files {5, 10, 15, 20}?	Sort: 5,10,15,20. Merge 5+10=15(sum=15). List: 15,15,20. Merge 15+15=30(sum=45). List: 20,30. Merge 20+30=50(sum=95). Total=95. Optimal is also 95 here.

PROGRAM 17

Huffman Code using Tree and Linked List (Greedy Approach)

17.1 Aim

To implement Huffman Encoding using a Greedy Algorithm with Binary Tree and Min-Heap (Priority Queue) to generate optimal variable-length prefix codes.

17.2 Theory

Huffman Coding assigns shorter binary codes to more frequent characters and longer codes to less frequent ones, minimizing total encoded message length. It is a lossless data compression algorithm.

Greedy Choice: Always merge the two nodes with the smallest frequencies. Build the tree bottom-up from leaves to root.

The resulting binary tree is used to generate prefix-free codes: 0 for left branch, 1 for right branch. Prefix-free means no code is a prefix of another code.

A Min-Heap (Priority Queue) is used to efficiently extract the two minimum frequency nodes at each step, giving $O(n \log n)$ overall complexity.

17.3 Algorithm

```

Algorithm HUFFMAN(C, n)
Input : Characters with frequencies
Output : Huffman Codes
Step 1: Create a leaf node for each character.
Step 2: Insert all nodes in ascending order of frequency.
Step 3: While more than one node exists
    Remove two nodes having minimum frequency.
    Create new internal node.
    Frequency = left.freq + right.freq
    Insert new node back into list.
Step 4: Remaining node becomes root.
Step 5: Traverse tree
    Left edge -> 0
    Right edge -> 1
Step 6: Print codes.
End
  
```

17.4 Pseudocode (Horowitz-Sahani Style)

```

Algorithm Huffman(n)
{
    // f[i] = frequency of character i
    Initialize min-heap H with n single-node trees;
    for i := 1 to n-1 do
    {
        p := getMin(H); // Remove minimum frequency node
    }
  }
  
```

```

q := getMin(H); // Remove next minimum frequency node
r := new node;
r.lchild := p; r.rchild := q;
r.freq := p.freq + q.freq;
insert(H, r);
}
return getMin(H); // Root of Huffman Tree
}

```

17.5 Sample Input / Output

Input: n=6
 Characters: a b c d e f
 Frequencies: 45 13 12 16 9 5
 Huffman Codes:
 a : 0
 c : 100
 b : 101
 f : 1100
 e : 1101
 d : 111

17.6 Time & Space Complexity

Case	Time Complexity	Space Complexity
Building Min-Heap	$O(n \log n)$	$O(n)$
n-1 Extractions	$O(n \log n)$	$O(n)$
Overall	$O(n \log n)$	$O(n)$
Complexity	Time: $O(n \log n)$ for building tree Space: $O(n)$ for heap	

17.7 MCQs

#	Question	Options	Answer
1	Huffman Coding is used for:	A) Sorting data B) Lossless data compression C) Lossy image compression D) Encryption	B
2	What is the Greedy choice in Huffman Coding?	A) Always select highest frequency node B) Always merge two lowest frequency nodes C) Assign codes alphabetically D) Sort characters by ASCII value	B
3	In Huffman tree, left branch represents code:	A) 1 B) 2 C) 0 D) -1	C
4	What does the compare struct do in the C++ code?	A) Sorts by frequency descending B) Creates a min-heap (smallest frequency at top) C) Compares character data D) Balances the tree	B
5	What is a prefix-free code?	A) All codes have same length B) No code is a prefix of another code C) Codes start with 0 only D) Codes are unique integers	B
6	Internal nodes in Huffman tree have data = :	A) '\$' (dollar sign, placeholder) B) The most frequent character C) NULL D) Sum of frequencies as char	A
7	Time complexity of Huffman Coding is:	A) $O(n^2)$ B) $O(n)$ C) $O(n \log n)$ D) $O(\log n)$	C
8	In the sample output, 'a' gets code '0' because:	A) 'a' has the smallest frequency B) 'a' has the highest frequency (45) C) 'a' is the first character D) Alphabetical assignment	B

9	What is the root of the Huffman tree?	A) Character with highest frequency B) An internal node with total sum of all frequencies C) Character 'a' always D) The last merged node	B
10	Priority Queue in C++ is by default a:	A) Min-heap B) Max-heap C) Sorted array D) Binary search tree	B

17.8 Short Questions & Answers

#	Question	Answer
1	What is Huffman Coding?	Huffman Coding is a greedy algorithm for lossless data compression. It assigns variable-length binary codes to characters — shorter codes to more frequent characters — minimizing total encoded message length.
2	What is a prefix-free code and why is it important?	A prefix-free code (also called prefix code) ensures no codeword is a prefix of another. This allows unique decoding without separators. Huffman codes are always prefix-free because each character is a leaf in the tree.
3	Why does the most frequent character get the shortest code?	In Huffman's greedy strategy, the two lowest frequency nodes are merged first. High-frequency characters remain near the root longer, resulting in shorter paths (fewer bits) from root to leaf.
4	What does printCodes() do?	printCodes() recursively traverses the Huffman tree. It appends '0' when going left and '1' when going right. When it reaches a leaf node (character node), it prints the character and accumulated code string.
5	How does priority_queue with compare struct create a min-heap?	By default, priority_queue is a max-heap. The custom compare struct with operator() returning $l \rightarrow \text{freq} > r \rightarrow \text{freq}$ inverts the comparison, making the element with smallest frequency appear at the top — creating a min-heap.

17.9 Numericals

#	Problem	Solution
1	Characters: a(45), b(13), c(12), d(16), e(9), f(5). Build Huffman tree and find average code length.	Merge e(9)+f(5)=14. Merge c(12)+b(13)=25. Merge 14+d(16)=30. Merge 25+30=55. Merge a(45)+55=100. Codes: a=1bit, others=3-4bits. Avg = $(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) / 100 = 2.24$ bits/char
2	If original message has 1000 characters with above frequencies, what is total encoded bits?	Using Huffman: a($45 \cdot 1=45$), b($13 \cdot 3=39$), c($12 \cdot 3=36$), d($16 \cdot 3=48$), e($9 \cdot 4=36$), f($5 \cdot 4=20$). Total = 224 bits for 100 chars. For 1000 chars: 2240 bits. Fixed 3-bit: 3000 bits. Savings = 25%.
3	4 characters: A(40), B(35), C(15), D(10). Find Huffman codes.	Merge D(10)+C(15)=25. Merge 25+B(35)=60. Merge 60+A(40)=100. A:0, B:10, C:110, D:111. Or A:1, B:01, C:001, D:000 depending on tie-breaking.

Huffman Coding — a=45, b=13, c=12, d=16, e=9, f=5

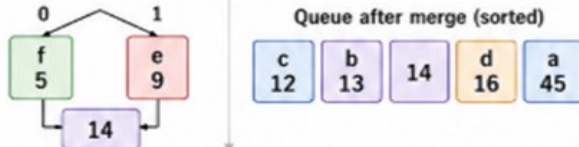
Frequencies

a	b	c	d	e	f
45	13	12	16	9	5

Sorted by frequency (ascending)

f = 5, e = 9, c = 12, b = 13, d = 16, a = 45

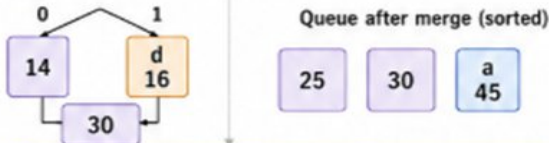
Step 1 Merge f(5) + e(9) = 14



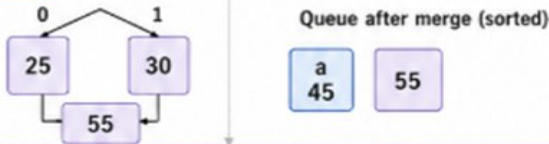
Step 2 Merge c(12) + b(13) = 25



Step 3 Merge 14 + d(16) = 30



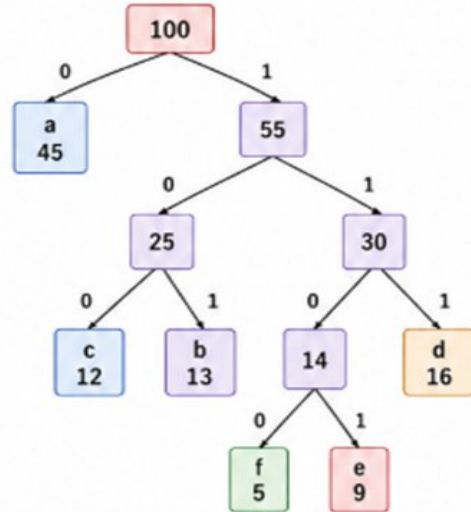
Step 4 Merge 25 + 30 = 55



Step 5 Merge a(45) + 55 = 100 (Root!)



Huffman Tree (0 = left, 1 = right)



Huffman Codes

Character	Frequency	Code	Length
a	45	0	1 (shortest)
b	13	101	3
c	12	100	3
d	16	111	3
e	9	1101	4
f	5	1100	4 (longest)

Average Code Length

Total frequency = 45 + 13 + 12 + 16 + 9 + 5 = 100

$$\text{Avg. code length} = \frac{(45 \times 1) + (13 \times 3) + (12 \times 3) + (16 \times 3) + (9 \times 4) + (5 \times 4)}{100} = \frac{45 + 39 + 36 + 48 + 36 + 20}{100} = \frac{224}{100} = 2.24 \text{ bits/char}$$

Fixed-length (3-bit) codes would use 3.00 bits/char (since $2^3 = 8 \geq 6$ symbols)

Huffman saves
3.00 - 2.24 = 0.76 bits/char
≈ 25% savings!

Sorted (by code): a:0 c:100 b:101 d:111 f:1100 e:1101